

Scientific Machine Learning (SciML)

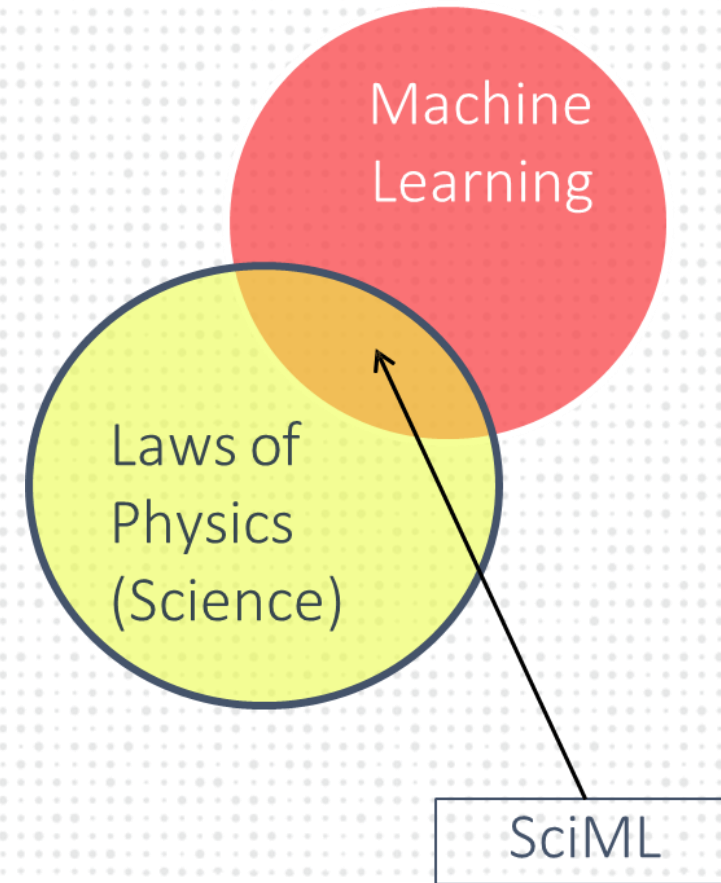
Module 1: Physics Informed Neural Networks

Topics

- Intro to Scientific Machine Learning (SciML)
- Finite difference implementation of partial differential equations (PDEs)
- **Live:** 1D heat equation implementation finite difference
- Physics-informed neural networks (PINNs)
- **Live:** 1D heat equation implementation using PINNs
- **Live:** Challenges and limitations of PINNs

Scientific Machine Learning (SciML)

“Scientific machine learning (SciML) is a core component of artificial intelligence (AI) and a computational technology that can be trained, with scientific data, to augment or automate human skills” – *Department of Energy*



Partial Differential Equation (PDE)

- A differential equation is a relationship among partial derivatives of a function u with respect to one or more independent variable(s) (x, t)

$$\frac{\partial}{\partial x} u(x, t) = u_x$$

General form of PDEs

- General form: $D(u) = 0$
 - For heat equation, $D = \frac{\partial}{\partial t} - \alpha \frac{\partial^2}{\partial x^2}$
 - $u(x, t)$: temperature at position x and time t ; α : diffusivity

Name	Equation	Application
Wave Equation	$u_{tt} = c^2 u_{xx}$	Vibrating String
Diffusion Equation	$u_t = \alpha u_{xx}$	Heat Flow
Laplace's Equation	$u_{xx} + u_{yy} = 0$	Electrostatics
Burgers Equation	$u_t + uu_x = Du_{xx}$	Fluid Mechanics
Cahn-Hilliard Equation	$u_t = (u^3 - u - u_{xx})_{xx}$	Phase Separation

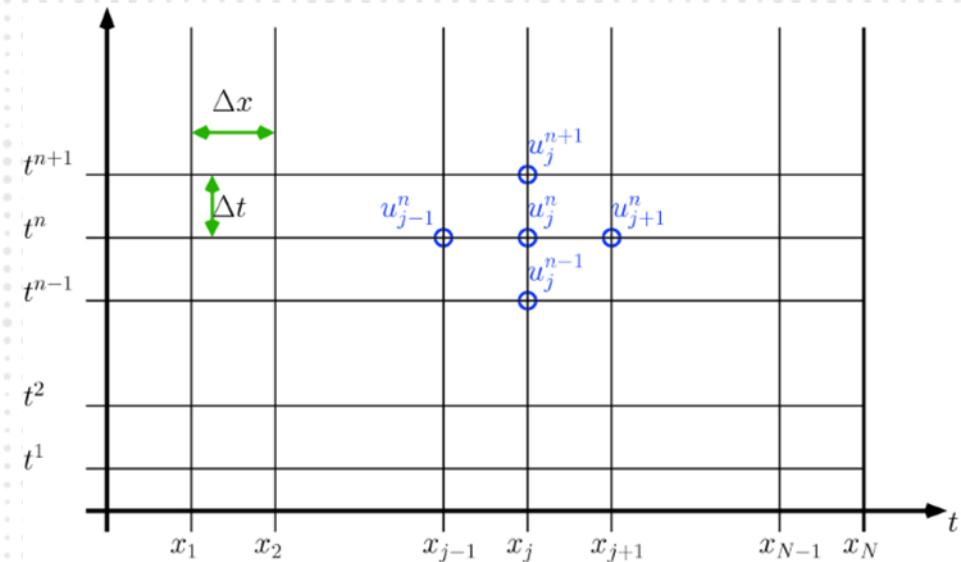
Finite Difference Method

- Based on Taylor series expansion

$$u(x, t + \Delta t) = u(x, t) + \Delta t \frac{\partial u}{\partial t} + O(t^2)$$

- $\frac{\partial u}{\partial t} \approx \frac{u(x, t + \Delta t) - u(x, t)}{\Delta t}$

- $\frac{\partial^2 u}{\partial x^2} \approx \frac{u(x - \Delta x, t) + u(x + \Delta x, t) - 2u(x, t)}{\Delta x^2}$



Heat Equation Discretization

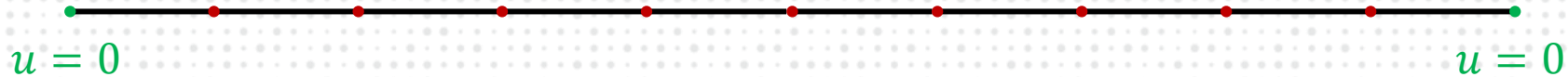
$$\frac{u(x, t + \Delta t) - u(x, t)}{\Delta t} = \alpha \frac{u(x - \Delta x, t) - 2u(x, t) + u(x + \Delta x, t)}{\Delta x^2}$$

$$u(x, t + \Delta t) = u(x, t) + \left(\frac{\alpha \Delta t}{\Delta x^2} \right) [u(x - \Delta x, t) + u(x + \Delta x, t) - 2u(x, t)]$$

- Solution is computed iteratively for each grid point (x_i, t_n) moving forward in time

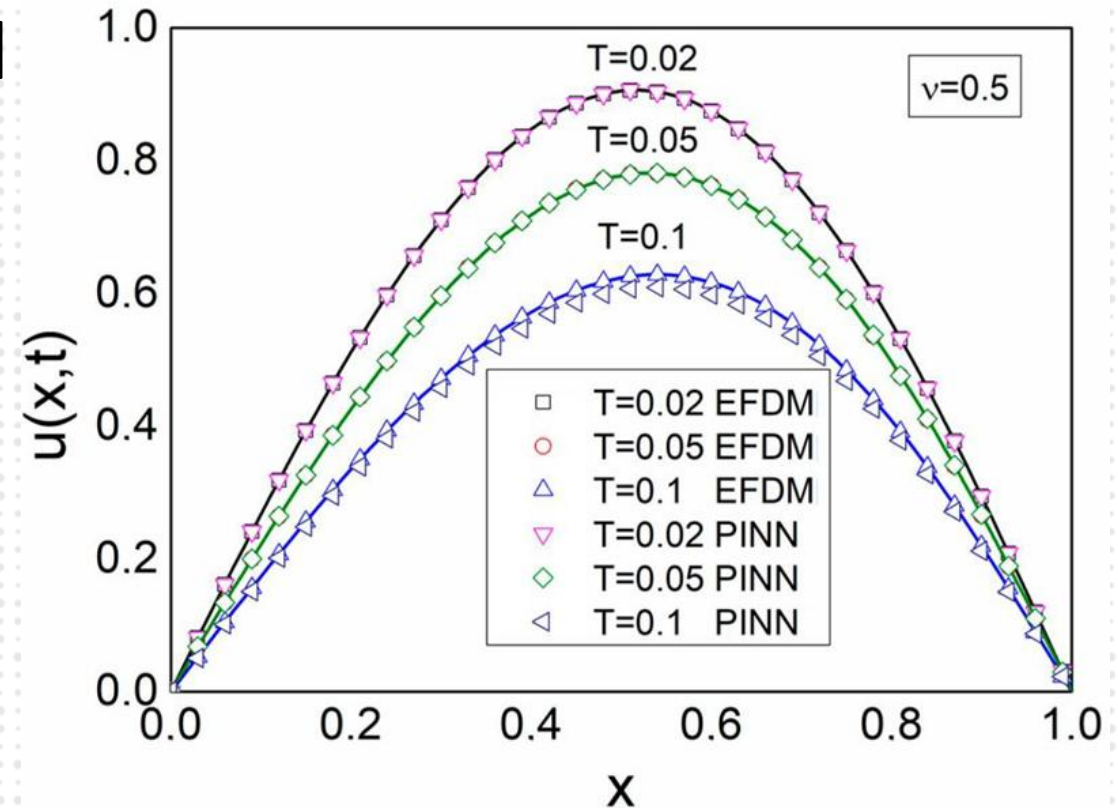
1D Heat Equation – Finite Difference

- Analytical solution: $u_{exact} = e^{-\alpha\pi^2 t} \sin(\pi x)$
- Domain: $\Omega = [0,1]; t = [0,1]$
- BCs: $u(x = 0, t) = u(x = 1, t) = 0$
- ICs: $u(x, 0) = \sin(\pi x)$



PINNs

- Why?: Need an accurate and fast solution of PDEs
- Can use neural network
- Unsupervised machine learning approach to solve PDEs



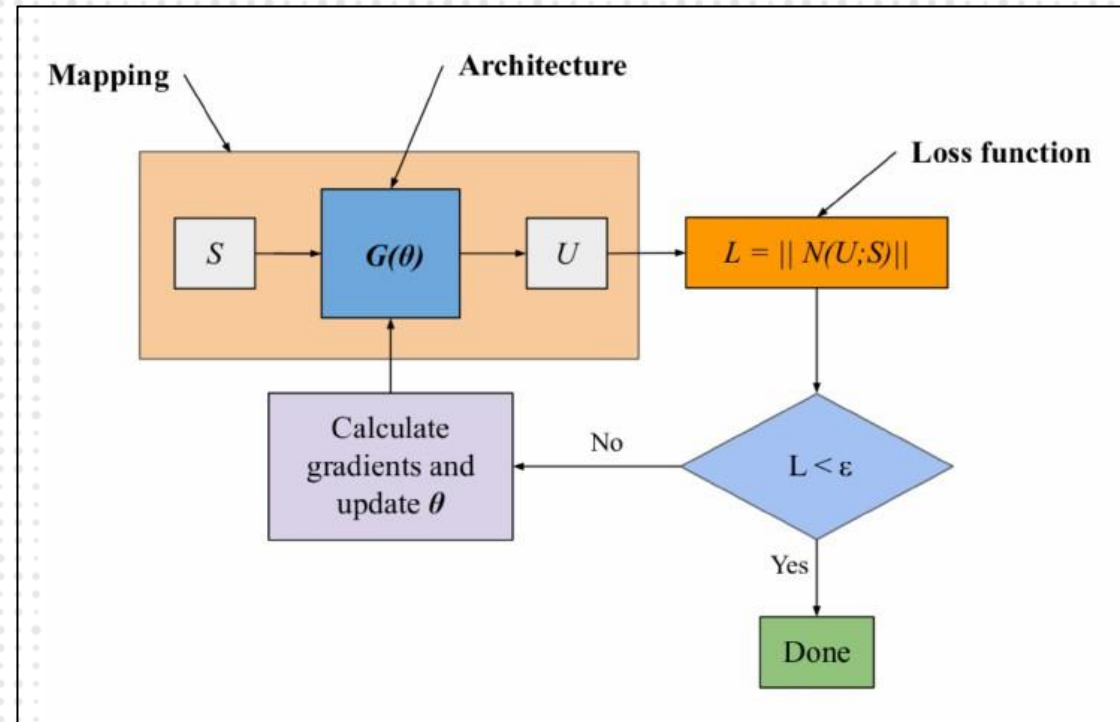
Savović, Svetislav, Miloš Ivanović, and Rui Min. "A Comparative Study of the Explicit Finite Difference Method and Physics-Informed Neural Networks for Solving the Burgers' Equation." *Axioms* 12.10 (2023): 982.

Physics-Informed Neural Networks

- Forward problems: Predict the behavior with known physics
 - Heat equation
- Inverse problems: Calculate parameters based on data
 - Uniaxial tensile testing
- Equation discovery

Workflow

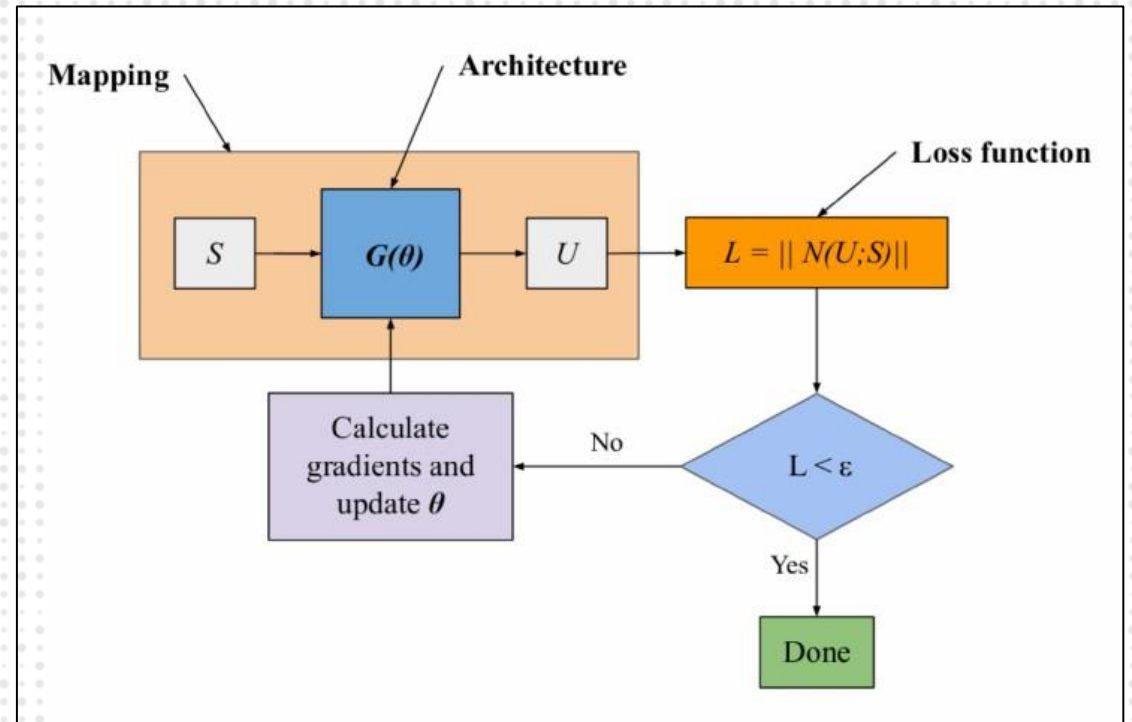
- Replace numerical methods using neural network (NN)
- Train NN to learn mapping from (x, t) to $u(x, t)$ using:
 1. PDE
 2. ICs, BCs, forcing
 3. Data, if available



Khara, Biswajit, et al. "Neufenet: Neural finite element solutions with theoretical bounds for parametric pdes." *Engineering with Computers* (2024): 1-23.

Advantages

- Mesh free
- Physics loss is included to ensure the solution confirms to PDE
- BC/IC loss is included to ensure the solution is unique

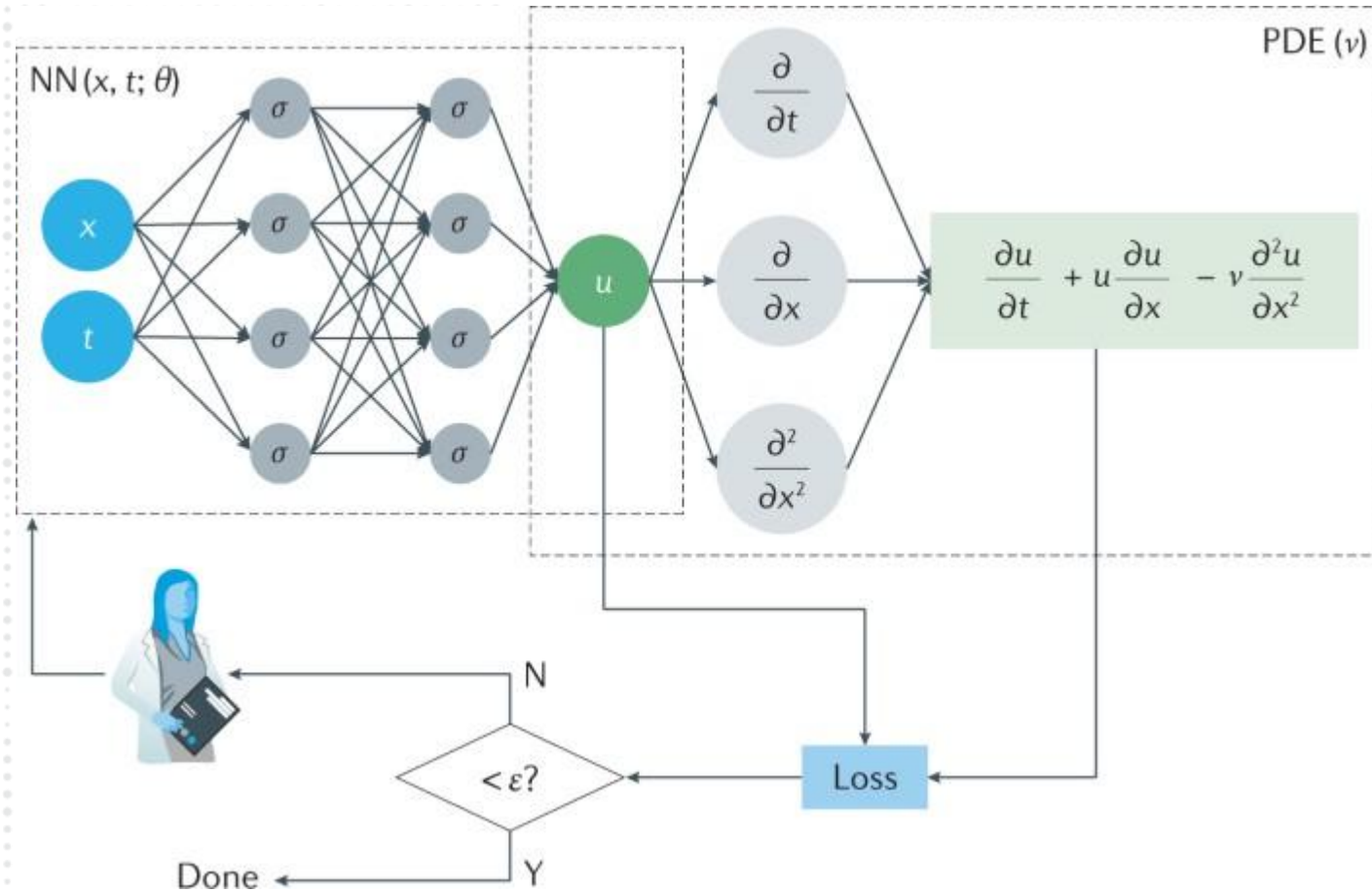


Khara, Biswajit, et al. "Neufenet: Neural finite element solutions with theoretical bounds for parametric pdes." *Engineering with Computers* (2024): 1-23.

Loss Function

- PDE: $D(u) = 0$
- BCs: $u(x = 0, t) = u(x = 1, t) = u_b$
- ICs: $u(x, t = 0) = f(x)$
- Loss:
 - $L_{PDE} = (D(u) - 0)^2$
 - $L_{BC} = (u(x = 0, t) - u_b)^2 + (u(x = 1, t) - u_b)^2$
 - $L_{IC} = (u(x, t = 0) - f(x))^2$
 - $L = L_{PDE} + \beta_1 L_{BC} + \beta_2 L_{IC}$

Computational Graph



Dieva, N., Aminev, D., Kravchenko, M., and Smirnov, N. "Overview of the Application of Physically Informed Neural Networks to the Problems of Nonlinear Fluid Flow in Porous Media." *Computation*, 2024

Training Loop

1. Sample collocation points
2. Compute NN outputs
3. Compute gradients w.r.t. input
4. Compute loss
5. Compute gradient of loss function w.r.t. parameters
6. Take gradient descent step

```
# PINN training psuedocode

#2.
t.requires_grad_(True) # tells PyTorch to start tracking graph
theta.requires_grad_(True)
u = NN(t, theta)

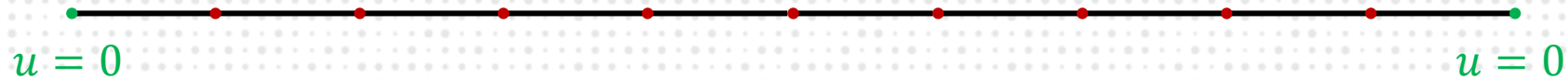
#3.
dudt = torch.autograd.grad(u, t, torch.ones_like(u),
                           create_graph=True)[0]
d2udt2 = torch.autograd.grad(dudt, t, torch.ones_like(u),
                             create_graph=True)[0]

#4.
physics_loss = torch.mean((m*d2udt2 + mu*dudt + k*u)**2)
loss = physics_loss + lambda_*boundary_loss

#5.
dtheta = torch.autograd.grad(loss, theta)[0]
```

1D Heat Equation – PINNs

- Analytical solution: $u_{exact} = e^{-\alpha\pi^2 t} \sin(\pi x)$
- Domain: $\Omega = [0,1]; t = [0,1]$
- BCs: $u(x = 0, t) = u(x = 1, t) = 0$
- ICs: $u(x, 0) = \sin(\pi x)$



THANK YOU